ENFOS

ENGINEERING · CODING TAKE-HOME

Reporting Landing Page Assessment

Build a full-stack reporting portal: a React frontend backed by a Java API, where users browse available reports and explore their data in interactive tables.

ROLE FOCUS

Full-Stack (React + Java)

SUBMISSION

Repo + README + Demo

1 Overview & Goal

You will build a **reporting landing page** where users can browse and interact with a set of available reports. The goal is a small but complete full-stack application that demonstrates clean architecture, thoughtful UI states, and a well-structured API.

The finished application should deliver:

- A **React** frontend built with **modern, modular, scalable components**
- A **Java** backend API (Spring Boot preferred)
- Three available reports: **Users**, **Departments**, and **Projects**

> **Scenario.** You are building an internal reporting portal. Users land on a reporting homepage, see the available reports, and open each one to view its data in a table. Treat this as a realistic slice of a production internal tool.

2 Frontend Requirements

Stack: React · modern, modular, scalable components

Landing page

Design a reporting landing page where users can browse the available reports and open any one of them. **How it looks and feels is up to you.** We want to see your take on what makes a good reporting home. At a minimum it should let users understand what reports exist, find a specific one easily, and get into a report.

You have full creative freedom over layout, visual style, and how each report is presented (cards, list, grid, or anything else). Useful context to surface per report might include a name, a short description, and when it was last updated, but the choices are yours.

Report detail / table view

- A dedicated table view for each report that renders its tabular data
- Clear **loading**, **empty**, and **error** states
- A way to navigate back to the landing page

3 Reports & Columns

Each report should display, at minimum, the columns below. You may add sensible additional columns if it improves the experience.

Users

People in the system

- User ID
- Name
- Email
- Role
- Status
- Created Date

Departments

Org structure

- Department ID
- Department Name
- Manager
- Employee Count
- Location

Projects

Active & past work

- Project ID
- Project Name
- Department
- Owner
- Status
- Start / End Date

4 Backend Requirements

Use **Java** for the backend; **Spring Boot** is preferred. Expose REST APIs to support the frontend. The backend **may use in-memory mock data**; a database is optional.

Required endpoints

Method	Endpoint	Returns
`GET`	`/api/reports`	List of available reports (metadata)
`GET`	`/api/reports/users`	Users report rows
`GET`	`/api/reports/departments`	Departments report rows
`GET`	`/api/reports/projects`	Projects report rows

5 Build & Deploy

Provide a **single command to build and deploy your application**. Everything required to run the application should be provided by you, so a reviewer can bring up the full stack from a clean checkout without hunting for missing pieces.

- A documented command (for example a script, a `make` target, or a `docker compose` command) that builds and runs the app
- All dependencies, configuration, and environment setup included or clearly scripted
- Both the frontend and the Java backend should start from that single command
- Note any prerequisites (such as required runtime versions) in the README

6 Expected Functionality

A user should be able to:

- ✓ View all available reports on the landing page
- ✓ Search / filter reports by name
- ✓ Open a report and view its data in a table
- ✓ See loading and error states while data is fetched
- ✓ Navigate back to the landing page

7 Evaluation Criteria

We review submissions across the following areas. There is no single “right” solution. We are most interested in how you reason about structure, state, and user experience.

Area	What we look for
Code organization	Readable, well-structured code with sensible separation of concerns
React structure	Sensible component breakdown, state management, and data fetching
Component design	Modern, modular, scalable components with effective, consistent styling
API & backend design	Clean REST design and a well-organized Java/Spring Boot structure
State handling	Thoughtful loading, empty, and error states
Responsive layout	Works well across viewport sizes
Overall UX	The portal feels coherent and pleasant to use

8 Use of AI Tools

> Feel free to use any **AI tools, code generators, documentation, libraries, or other productivity tools** while completing this assessment.
>
> However, you should be able to **explain your solution, design decisions, and implementation details** during a follow-up discussion or interview.

In short: use whatever helps you work effectively, but own and understand the result. We care far more about your reasoning than about whether a tool helped you type it.

9 Submission

Please submit

- ✓ **GitHub repository link** or zipped source code
- ✓ **README** with setup and run instructions for both frontend and backend
- ✓ **Screenshots** or a short demo video
- ✓ **Notes** on your assumptions and tradeoffs

Enfos · Engineering Take-Home Assessment: Reporting Landing Page. Questions about the assessment are welcome before you begin.